Learning-Based Sparsification of Dynamic Graphs in Robotic Exploration Algorithms

1st Adithya V. Sastry
Independent Researcher
Knoxville, TN, USA
avs.origami@gmail.com

2nd Bibek Poudel
University of Tennessee, Knoxville
Knoxville, TN, USA
bpoudel3@vols.utk.edu

3rd Weizi Li
University of California, Riverside
Riverside, CA, USA
weizi.li@ucr.edu

***Abstract*—Many robotic exploration algorithms rely on graph structures for frontier-based exploration and dynamic path planning. However, these graphs grow rapidly, accumulating redundant information and impacting performance. We present a transformer-based framework trained with Proximal Policy Optimization (PPO) to prune these graphs during exploration, limiting their growth and reducing the accumulation of excess information. The framework was evaluated on simulations of a robotic agent using Rapidly Exploring Random Trees (RRT) to carry out frontier-based exploration, where the learned policy reduces graph size by up to 96%. We find preliminary evidence that our framework learns to associate pruning decisions with exploration outcomes despite sparse, delayed reward signals. We also observe that while intelligent pruning achieves a lower rate of exploration compared to baselines, it yields the lowest standard deviation, producing the most consistent exploration across varied environments. To the best of our knowledge, these results are the first suggesting the viability of RL in sparsification of dynamic graphs used in robotic exploration algorithms.**

***Index Terms*—robotic exploration, graph sparsification, deep reinforcement learning**

I. Introduction

Autonomous systems have achieved remarkable progress in navigating complex, unstructured environments, enabling robots to carry out tasks that were previously limited to human operators [1]. Search and rescue, space exploration, and surveillance all stand to benefit from autonomous robotic exploration, as these tasks often require operation in environments too hazardous or inaccessible for humans [2]. However, many exploration algorithms rely on graph or tree structures for path planning and waypoint creation [3], [4], and these structures grow continuously as the agent explores, accumulating redundant information that can degrade performance.

By pruning these graphs, we can allow a robot to focus on pushing frontiers with greater potential for information gain, reducing the overhead of maintaining large, dense structures. Yet, the relationships between the exploration graph and the environment shift as the robot gains new information about its surroundings [3]–[5]. Pruning decisions must therefore be made sequentially under partial observability, with rewards that are delayed and difficult to attribute to individual actions. These characteristics make reinforcement learning (RL) a promising tool for learning an effective pruning strategy, as it is designed to handle sequential decision-making with sparse, delayed feedback and without supervised labels.

Fig. 1. **Left:** An example environment the robot explores. **Middle:** Exploration progress without pruning. **Right:** Exploration progress with pruning over a comparable time period. Magenta blocks are obstacles, gray area represents explored free space, black area represents unexplored free space, blue represents the exploration graph, and yellow represents the robot's field of view. Pruning reduces the exploration graph size by up to 96%, demonstrating the potential for intelligent sparsification in graph-based exploration.

We introduce a transformer-based framework that utilizes RL to prune dynamic graphs used in frontier-based robotic exploration, reducing the size of exploration graphs by 96% (Fig. 1). Preliminary results show that while intelligent pruning currently reduces the final explored area compared to baselines (42.84% for intelligent pruning, versus 71.33% and 70.99% for randomly pruned and unpruned baselines, respectively) given a fixed exploration time, it also yields the most consistent exploration across varied environments ($\pm$7.76%) compared to baselines ($\pm$10.47% and $\pm$17.99% for randomly pruned and unpruned baselines, respectively). To the best of our knowledge, these results are the first to suggest the viability of RL for intelligent sparsification of dynamic graphs in graph-based robotic exploration algorithms. As such, this research finds learned policies to be a novel tool for enabling robots to carry out such tasks with reduced or sparse information. Source code can be found in the GitHub repository[1].

II. Related Work

Frontier-based exploration [6] is a widely used paradigm in which graph or tree structures are used to identify frontiers (the boundary between explored and unexplored regions) and guide pathfinding towards these regions, continuously maximizing information gain [3], [5], [7]. Deep reinforcement learning (DRL) also sees extensive use in robotic exploration [8], with methods ranging from using direct policy learning to identify exploration targets and carry out exploration [9], [10], to using graph neural networks to make navigation

[1] https://github.com/avs-origami/graphsparse

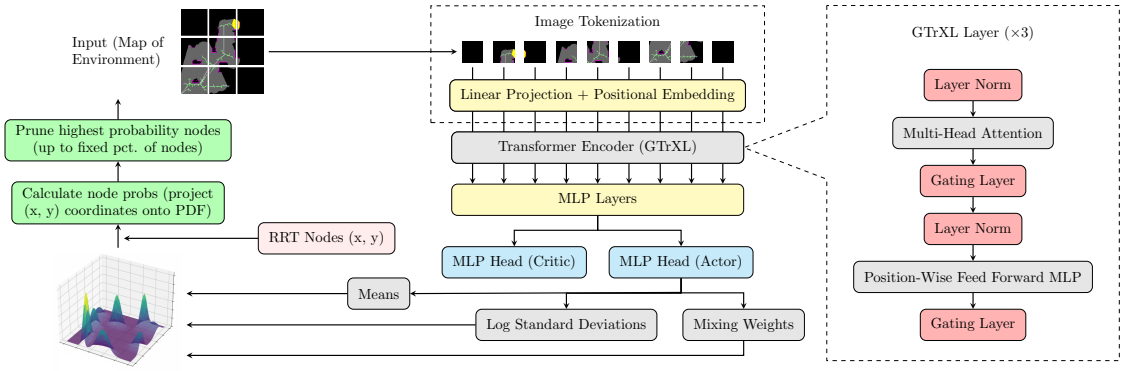

Fig. 2. Overview of the framework for pruning robotic exploration graphs. An image of the current map (top left) is split into patches, tokenized, and enriched with positional embeddings before passing through a Gated Transformer-XL encoder and multi-layer perceptron (MLP) layers for feature extraction. The extracted features feed into separate actor and critic heads, where the actor predicts the means, standard deviations, and mixing weights that define a Gaussian Mixture Model (GMM) over the environment. To determine which nodes to prune, the coordinates of nodes in the exploration graph are projected onto this GMM, and the highest-probability nodes are removed.

decisions based on specialized graph representations of the environment [11]–[14]. Canonical algorithms may be better suited to rapid replanning or dynamic environments, while DRL approaches can outperform them in more structured settings [15]. However, purely learned approaches must relearn basic exploration behaviors from scratch and can struggle when encountering diverse environments. Rather than using a learned policy directly for exploration, we combine a canonical exploration algorithm with DRL for the subtask of pruning the exploration graph. This preserves the algorithmic robustness of frontier-based methods while using RL to learn the complex, evolving relationship between the graph and the environment, thereby enabling exploration with reduced information.

The pruning task requires the model to learn how sequences of decisions affect future exploration over long horizons. To capture these long-term dependencies, we rely on the Gated Transformer-XL (GTrXL) [16]. Transformers [17] are effective for feature representation across a range of tasks [18]–[20], and recent works extend their capacity for long-term memory [21], [22]. GTrXL adds gating mechanisms that stabilize training under the non-stationary data distributions typical in RL, making it well-suited for our task where the state and action spaces shift as exploration progresses.

In short, while existing studies address robotic exploration through either purely algorithmic strategies [3]–[7] or fully learned approaches [8]–[13], we introduce a hybrid approach that leverages RL to perform intelligent sparsification on the exploration graph. This preserves the robustness, adaptability, and interpretability of algorithmic methods while leveraging learned policies to reduce redundant information.

III. Methodology

We define the task as pruning nodes from the exploration graph of a frontier-based robotic exploration algorithm, with the objective of producing a sparse graph that is at least as effective at enabling exploration as the original. To simplify the problem, we assume a fixed, known environment size, which enables fixed-size inputs and outputs to the model. Our solution framework is outlined in Fig. 2.

A. Markov Decision Process

We formulate the graph pruning strategy as a partially observable Markov decision process $(S, A, T, R, \Omega, O, \gamma)$, where S is the set of environment states, A is the set of actions, $T : S \times A \times S \to [0, 1]$ is the probabilistic state transition function, and $R : S \times A \to \mathbb{R}$ is the reward function. Due to partial observability, Ω is the set of observations, $O : S \times A \times \Omega \to [0, 1]$ is the observation probability function, and $\gamma \in [0, 1)$ is the discount factor. At each timestep, the agent observes an incomplete view of the environment and selects a pruning action to maximize the cumulative discounted reward $G_t = \sum_{k=t}^{T} \gamma^{k-t} r_k$. We train the policy with Proximal Policy Optimization (PPO) [23] and Generalized Advantage Estimation [24], as PPO is effective for problems with delayed or sparse reward signals [25].

State. In contrast to approaches which rely on specialized graph representations of the environment [11]–[14], the state is an image of the current environment map with the exploration graph as an overlay, eliminating additional processing to encode spatial relationships between the exploration graph and the environment. The image is tokenized following the Vision Transformer [20] where it is split into fixed-size patches, linearly embedded, and combined with positional embeddings. Self-attention then integrates information from across the whole image, unlike other methods like convolutional neural networks, which consider only local context [20].

Action. Since the set of nodes in the exploration graph is constantly changing, the model cannot directly select nodes to prune. Instead, we take advantage of the spatial regularity of the environment to parameterize the action space to a fixed size. Given a Gaussian Mixture Model (GMM) [26],

$$p(x) = \sum_{k=1}^{K} \pi_k \mathcal{N}(\mathbf{x} | \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \tag{1}$$

our framework predicts the means $\boldsymbol{\mu}_k$, standard deviations $\boldsymbol{\sigma}_k$ (used to construct diagonal covariance matrices $\boldsymbol{\Sigma}_k$), and mixing coefficients π_k. The means represent coordinates, creating a smooth probability function over the environment

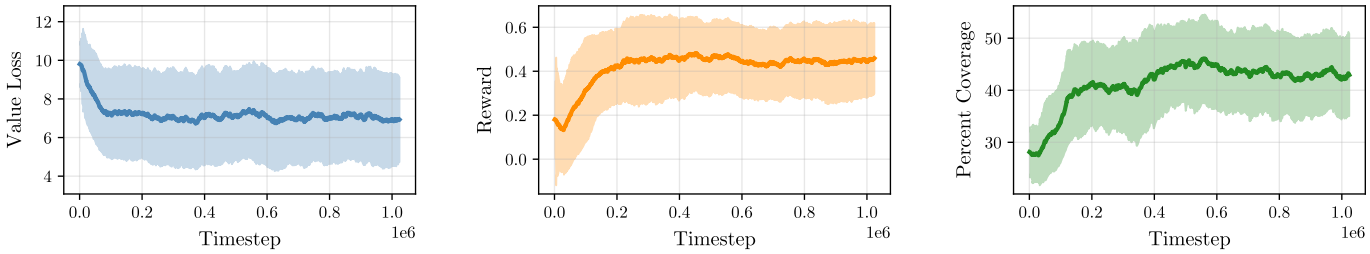

Fig. 3. **Left:** value loss during training. **Middle:** average reward per episode. **Right:** average percent coverage of the environment per episode. These curves are sampled from one training cycle; we observed similar results across multiple cycles using the same parameters. Reward stabilizes at 0.45 and coverage reaches 45% by 600k timesteps, suggesting the model learns a pruning policy over time.

that assigns pruning probabilities to nodes based on spatial relationships. Alternative approaches we tried for predicting over a variable-sized action space are detailed in Appendix A.

Reward. The reward is calculated in three stages. For each pruned node i, we assign two component rewards based on the type and location of the node:

$$r_f(i) = 1 - 2 \cdot \mathbb{1}_{i \,\in\, \text{frontiers}}, \tag{2}$$

$$r_c(i) = 1 - 2 \cdot \mathbb{1}_{i \,\in\, \text{leaves} \,\cup\, \text{splits}}, \tag{3}$$

where "frontiers" is the set of nodes located on the frontier, "leaves" is the set of nodes without children, and "splits" is the set of nodes with multiple children. That is, each component is -1 if the pruned node satisfies its condition, and $+1$ otherwise. Next, the timestep reward is calculated as the average of all per-node rewards for that timestep, minus a penalty for the difficulty of finding a suitable next move after pruning, discouraging the model from taking overly destructive actions:

$$R_t = \frac{1}{|P|}\left(\sum_{i \in P} r_f(i) + r_c(i)\right) - \frac{\lambda_A N_a}{2\lambda_M}, \tag{4}$$

where P ($|P| \geq 1$) is the set of nodes pruned in that timestep, N_a is the number of attempts to find a suitable move, $\lambda_A = 5$ is the attempt penalty scaling factor, and λ_M is the maximum number of moves before episode termination; see Table II. Finally, terminal states receive a large bonus reward,

$$B = \lambda_B \cdot (e^C - 1), \tag{5}$$

where C is the fraction of the environment that has been mapped once the terminal state is reached and $\lambda_B = 8$ is the scaling factor for the terminal reward. The complete reward is then defined as $R = R_t + \mathbb{1}_{\text{terminal}} \cdot B$. The terminal bonus grows exponentially with final explored area, strongly incentivizing complete exploration, and the inclusion of timestep rewards helps with credit assignment given the sparse reward signal (discussed further in Appendix A).

IV. Experiments

Setup. We developed a custom 2D simulation (depicted in Fig. 1) to conduct all experiments. It consists of a pixel grid randomly populated with obstacles, similar to the evidence grids originally seen in frontier-based exploration [6].

The simulation is designed to help us narrow the focus of this work through a set of simplifying assumptions. First, the size of the environment is fixed and known. In a practical application, this means a robot using our model has an upper limit on the area it can explore; however, real applications requiring exploration of a large area would likely already rely on multiple robots to cover the area more efficiently. Second, while a physical robot would have sensors that enable it to detect obstacles, the agent in the simulated environment has a field of view which serves this function. The simulation also disregards most uncertainty in the robot's position and in the map of the environment. In real-world applications, such uncertainty typically results from the accumulation of error in sensor measurements and Simultaneous Localization and Mapping (SLAM) systems.

The exploration algorithm we use for experiments relies on a Rapidly-exploring Random Tree (RRT) grown within the confines of the explored area to detect frontiers. Then, a frontier is selected based on proximity and other factors, and the robot moves towards it. This method shares similarities to other RRT-based exploration strategies [5], with the key difference of maintaining a global RRT rather than local RRTs.

The RL policy is trained across 10^6 timesteps on an Intel Core i9-14900K processor and an NVIDIA RTX A5000 GPU. Hyperparameters are listed in Table II. During training, the model predicts a pruning strategy as described in Section III-A. The number of pruned nodes is selected such that the RRT growth rate is reduced by a fixed fraction $\lambda_f = 0.96$ of its normal growth rate. Once the robot has taken a fixed number of moves or the RRT has been grown a fixed number of times (both hyperparameters; see Table II), the simulation is reset and obstacles are randomized.

Results. The results reported in Fig. 3 illustrate the model's learning progress. The value loss curve decreases to 50% of its original value within the first 75k timesteps. The reward curve, which begins at 0.18 at the start of training, stabilizes at an average of 0.45 within the first 400k timesteps. This is linked to a similar overall increase in the final percentage of the environment explored, stabilizing at 40% exploration by 400k timesteps and further increasing to 45% exploration by 600k timesteps. These training curves provide preliminary evidence that our framework is capable of learning the relationship between pruning decisions and the progression of exploration across highly varied environments.

Additionally, Table I reports the average percentage of the environment explored across simulations with randomized ob-

TABLE I

FINAL EXPLORED AREA UNDER DIFFERENT PRUNING STRATEGIES, GIVEN A FIXED TIME PERIOD AND $N = 600$ SIMULATIONS PER STRATEGY.

Method	Final progress (%)
No Pruning	70.99 $\pm$17.99
Random Pruning	71.33 $\pm$10.47
Intelligent Pruning	42.84 $\pm$7.76

Intelligent pruning achieves the lowest standard deviation compared to the baselines, indicating the most consistent exploration across varied environments despite lower mean coverage.

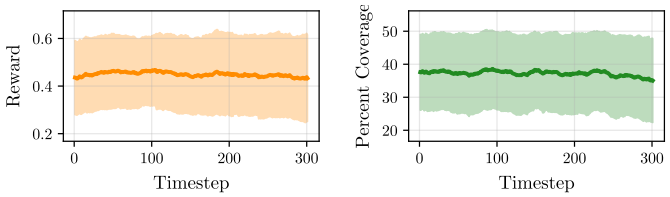

Fig. 4. Initial reward (**left**) and percent coverage (**right**), *before training*, when introducing noise to the Gaussian Mixture Model-assigned probabilities. The noisy variant achieves 2.5× the initial reward and 1.3× the initial coverage of the primary experiments before any training, suggesting the current parameterization of the action space may be a limiting factor.

stacle layouts ($N = 600$ complete simulations for each pruning strategy; a fixed number of timesteps per simulation; see Table II). The random pruning baseline replaces the learned GMM with uniform random probability assignment; all other pipeline components, pruning rates, and environment seeds are identical. Intelligent pruning exhibits the lowest rate of exploration, averaging 42.84% exploration given the same total exploration time as the baselines. However, it also results in the most consistent exploration performance across varied environments ($\pm 7.76\%$). We hypothesize that this performance gap relates to limitations of the GMM – creating a GMM expressive enough to represent optimal pruning strategies may require an impractically large number of components.

The comparable performance of exploration with random pruning and without pruning suggests that uniform random pruning may serve as a component of a more effective solution. We tested this idea by introducing noise to the GMM using trigonometric functions scaled to 10^{-3} times the maximum GMM probability across the environment and enabling the model to selectively activate or deactivate GMM components. In regions where GMM probabilities approach zero, the noise becomes significant, causing behavior akin to random pruning. By activating or deactivating components, the model can control whether each region uses learned or random pruning. We observed much higher initial exploration performance, as seen in Fig. 4. However, this represents a few isolated experiments, so more work is needed to confirm these results. Regardless, this suggests that performance can be improved by replacing the GMM with a different probability density function to allow for greater variation with fewer parameters.

Alternatively, performing coarse random pruning at each timestep before applying intelligent pruning [27] may make certain spatial relationships more apparent to the model by simplifying the input. However, this has not been tested.

TABLE II

HYPERPARAMETERS FOR PPO, POLICY NETWORKS, AND SIMULATION.

	Parameter	Value
PPO	Learning Rate (α)	3×10^{-4}
	Discount Factor (γ)	0.99
	GAE Estimation (λ)	0.95
	Clip Parameter (ϵ)	0.1
	Value Function Coeff.	0.5
	Entropy Coeff.	0.01
	Update Frequency	512
	K-epochs	4
	Num. Minibatch	4
	Target KL Divergence	0.03
Policy	GTrXL Layer Size	1024
	GTrXL Layers	3
	Attn. Heads	8
	Attn. Head Size	512
	PwFF Size	512
	GTrXL Mem. Len.	400
	Num. GMM Components	8
	Hidden Layers (Actor)	$[6400, 1600, 512, 512, 512, 512]$
	Hidden Layers (Critic)	$[6400, 1600, 512, 512, 512, 512]$
Sim.	Env. Dimensions	250px × 250px
	Max. Robot Moves (λ_M)	100
	Max. RRT Growth Attempts	100

V. DISCUSSION AND CONCLUSION

We introduce a transformer-based framework that learns to sparsify the exploration graphs of frontier-based robotic exploration algorithms, reducing graph size by up to 96%. The learning curves in Section IV provide preliminary evidence that the model associates pruning decisions with exploration outcomes despite a sparse, delayed reward signal. While intelligent pruning currently reduces the overall rate of exploration compared to baselines, it yields the lowest standard deviation in the rate of exploration, producing the most consistent exploration across varied environments. Random pruning performs comparably to exploration without pruning under the same 96% pruning rate, suggesting the resilience of frontier-based exploration to sparsification. If leveraged by a more effective learned policy, this resilience may enable us to effectively reduce computational overhead in graph-based exploration.

A key limitation is the lack of comparable methods; prior graph pruning work targets static graphs with supervised learning, which does not apply to the evolving graphs in this problem. As discussed in Section IV, the expressivity of the GMM may be a bottleneck, and replacing it with a more expressive probability density function is a promising direction for closing the performance gap. Validating these results on physical robots and in higher-clutter environments remains necessary, as the simulation makes simplifying assumptions about localization uncertainty and environment size. Overall, results suggest that RL-based sparsification of dynamic graphs in robotic exploration is feasible, providing a novel approach that can be refined and built upon, and opening new avenues for improving performance and reducing computational overhead in graph-based robotic exploration algorithms.

REFERENCES

[1] Jin Wang, Huan Yu, Zhi Zheng, Guodong Lu, Kewen Zhang, Tao Zheng, and Cong Fang. Autonomous robotic exploration with region-biased sampling and consistent decision making. *Complex & Intelligent Systems*, 9(5), Oct 2023.

[2] Héctor Azpúrua, Maíra Saboia, Gustavo M. Freitas, Lillian Clark, Ali akbar Agha-mohammadi, Gustavo Pessin, Mario F.M. Campos, and Douglas G. Macharet. A survey on the autonomous exploration of confined subterranean spaces: Perspectives from real-word and industrial robotic deployments. *Robotics and Autonomous Systems*, 160:104304, 2023.

[3] Tung Dang, Marco Tranzatto, Shehryar Masaud Khan Khattak, Frank Mascarich, Kostas Alexis, and Marco Hutter. Graph-based subterranean exploration path planning using aerial and legged robots. *Journal of Field Robotics*, 37(S 8):1363 – 1388, 2020-12.

[4] Indraneel Patil, Rachel Zheng, Charvi Gupta, Jaekyung Song, Narendar Sriram, and Katia Sycara. Graph-based simultaneous coverage and exploration planning for fast multi-robot search. *arXiv preprint arXiv:2303.02259*, 2023.

[5] Hassan Umari and Shayok Mukhopadhyay. Autonomous robotic exploration based on multiple rapidly-exploring randomized trees. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1396–1402, 2017.

[6] B. Yamauchi. A frontier-based approach for autonomous exploration. In *Proceedings of the 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation*, CIRA '97, page 146, USA, 1997. IEEE Computer Society.

[7] Hyejeong Ryu. Graph search-based exploration method using a frontier-graph structure for mobile robots. *Sensors*, 20(21), 2020.

[8] Zuxin Liu, Mohit Deshpande, Tony Qi, Ding Zhao, Rajasimman Madhivanan, and Arnie Sen. Learning to explore (l2e): Deep reinforcement learning-based autonomous exploration for household robot. 2023.

[9] Matteo Luperto, Luca Fochetta, and Francesco Amigoni. Exploration of indoor environments through predicting the layout of partially observed rooms. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '21, page 836–843, Richland, SC, 2021. International Foundation for Autonomous Agents and Multiagent Systems.

[10] Lukas Schmid, Chao Ni, Yuliang Zhong, Roland Siegwart, and Olov Andersson. Fast and compute-efficient sampling-based local exploration planning via distribution learning. *IEEE Robotics and Automation Letters*, 7:7810–7817, 07 2022.

[11] Fanfei Chen, John D. Martin, Yewei Huang, Jinkun Wang, and Brendan Englot. Autonomous exploration under uncertainty via deep reinforcement learning on graphs. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6140–6147, 2020.

[12] Fanfei Chen, Paul Szenher, Yewei Huang, Jinkun Wang, Tixiao Shan, Shi Bai, and Brendan Englot. Zero-shot reinforcement learning on graphs for autonomous exploration under uncertainty. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5193–5199, 2021.

[13] Tianze Luo, Budhitama Subagdja, Di Wang, and Ah-Hwee Tan. Multi-agent collaborative exploration through graph-based deep reinforcement learning. In *2019 IEEE International Conference on Agents (ICA)*, pages 2–7, 2019.

[14] Hao Zhang, Jiyu Cheng, Lin Zhang, Yibin Li, and Wei Zhang. H2gnn: Hierarchical-hops graph neural networks for multi-robot exploration in unknown environments. *IEEE Robotics and Automation Letters*, 7(2):3435–3442, 2022.

[15] Diego Arce, Jans Solano, and Cesar Beltrán. A comparison study between traditional and deep-reinforcement-learning-based algorithms for indoor autonomous navigation in dynamic scenarios. *Sensors*, 23(24), 2023.

[16] Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphael Lopez Kaufman, Aidan Clark, Seb Noury, et al. Stabilizing transformers for reinforcement learning. In *International conference on machine learning*, pages 7487–7498. PMLR, 2020.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Ł ukasz Kaiser, and Illia Polosukhin. Attention is all you need. In I. Guyon, U. Von Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 30. Curran Associates, Inc., 2017.

[18] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[19] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. Language models are unsupervised multitask learners. 2019.

[20] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. *CoRR*, abs/2010.11929, 2020.

[21] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. *arXiv preprint arXiv:1901.02860*, 2019.

[22] Tianwei Ni, Michel Ma, Benjamin Eysenbach, and Pierre-Luc Bacon. When do transformers shine in rl? decoupling memory from credit assignment. *Advances in Neural Information Processing Systems*, 36, 2024.

[23] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[24] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.

[25] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. *arXiv preprint arXiv:1810.12894*, 2018.

[26] Geoffrey McLachlan and David Peel. *General Introduction*. John Wiley & Sons, Ltd, 2000.

[27] Ryan Wickman. Sparrl: Graph sparsification via deep reinforcement learning. In *Proceedings of the 2022 International Conference on Management of Data*, SIGMOD '22, page 2521–2523, New York, NY, USA, 2022. Association for Computing Machinery.

[28] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks, 2018.

[29] Annamalai Narayanan, Mahinthan Chandramohan, Rajasekar Venkatesan, Lihui Chen, Yang Liu, and Shantanu Jaiswal. graph2vec: Learning distributed representations of graphs. *CoRR*, abs/1707.05005, 2017.

[30] Viacheslav Sinii, Alexander Nikulin, Vladislav Kurenkov, Ilya Zisman, and Sergey Kolesnikov. In-context reinforcement learning for variable action spaces. *arXiv preprint arXiv:2312.13327*, 2023.

Appendix A
Challenges

Due to the complexity of the problem, much of our work did not make it into the solution presented in this paper, and is therefore not discussed elsewhere. However, we believe it is valuable to document some of the major challenges and work done to overcome these challenges.

A. Graph Encoding

The biggest challenge in handling dynamically expanding graphs, such as those used in robotic exploration, is that the number of nodes in the graph is constantly changing. Therefore, these graphs cannot be passed as input to a model via traditional graph encodings, such as adjacency lists, adjacency matrices, edge lists, or incidence matrices, if the model must be trained using RL. We originally tried making use of a pretrained Graph Attention Transformer (GAT) [28] encoder to create fixed-size representations of variable-sized graphs, but designing an appropriate adjunct task to train the encoder was challenging, especially given the highly variable nature of the exploration graphs and the difficulty of discerning which features were most relevant to pruning decisions. More importantly, since all pruning decisions directly influence the structure of future input graphs, it was impractical to use a pretrained encoder when the inputs may look drastically different once training begins for the rest of the model. It would also not have been possible to train both the GAT encoder and the rest of the model simultaneously using traditional reinforcement learning techniques, since the input to the GAT encoder is of variable length, while reinforcement learning pipelines typically require all elements of the MDP to be of fixed size.

A second approach we explored was the use of Graph2Vec [29], a task-agnostic method that promises an unsupervised learning approach to encoding graphs of variable sizes while preserving structural equivalence (i.e. similar graphs produce similar embeddings). However, as with the GAT encoder, this would require pre-generating graphs to train the encoder on, and an encoder trained in this manner will likely be unable to properly represent the variety of graph structures that may arise over the course of training. Additionally, after working with this approach for some time, it became obvious that any graph encoding approaches that directly use the graph data would make it difficult to encode the environment features and the spatial relationships between the graph and the environment. Therefore, we settled on using an image to represent the exploration graph, environment features, and spatial relationships within a fixed size input, taking advantage of our simplifying assumption that the environment would always be of a fixed, known size.

B. Variable-size Action Spaces

Similar to the issue with graph encoding, the model cannot directly predict probabilities for each of the nodes in the graph because the graph is of a variable size. Therefore, an intermediate, fixed-size representation of the action is necessary to map to the variable-sized set of nodes. We originally intended to make use of a solution that relies on in-context reinforcement learning to match intermediate action embeddings to desired outcomes by allowing the model to experiment with actions to observe their effect on the environment, allowing the model to adapt in real-time to action spaces not seen during training [30]. However, this requires that changes in the action space occur infrequently, allowing the model time to build context regarding available actions. In this work, any action taken immediately modifies the set of available future actions, and this lack of an experimentation phase would prevent the model from establishing context on the effects of different action embeddings. As a result, we settled on the approach of using a probability distribution over the area of the environment to use the spatial regularity to encode actions as a fixed-size representation. Internally treating the GMM parameters as the action when training the model works since the mapping between a given set of GMM parameters and the resulting set of actions is deterministic for a given model input; i.e. for a particular environment map, a particular graph, and a particular set of GMM parameters, the set of nodes that is pruned will always be the same.

C. Credit Assignment and Delayed Rewards

One of the biggest challenges in enabling the model to learn an effective policy is overcoming the credit assignment problem and the delayed reward signal the model receives. The reward signal is significantly removed from the model's prediction – the model outputs parameters used to construct a GMM, which is then used to prune the current exploration graph, and the pruned exploration graph is then used to explore the environment further. However, the metric of performance is primarily the final coverage of the environment. So, for example, it does not matter how much coverage is gained from one exploration step to the next, as long as the environment is completely explored in the allotted time. Therefore, since the reward is significantly delayed and represents the accumulation of all actions taken over an episode, it is difficult for the model to determine which steps specifically led to the desired outcome. Combined with the fact that the state is a very information-rich but compact representation, this makes it very difficult for the model to learn an effective policy. We were able to partially remedy this issue by assigning small intermediate rewards based on factors that we knew to generally either inhibit or promote rapid exploration. While this does not fully solve the issue, it appears to give the model a clearer idea of which actions were beneficial and which were not, which improved the model's ability to learn a policy. However, this could also prevent the model from exploring all possible policies. Further work is needed to determine the best approach to tackle this challenge.

Acknowledgments

The authors would like to thank the Fluidic City Lab, the Tickle College of Engineering at the University of Tennessee Knoxville, and NVIDIA for their support of this research.